

LAPUTA — v2 MESH SCALING ADDENDUM

Enterprise Scale via Geometric Governance + The `evaluate_action` Chokepoint

Append to: LAPUTA_v1_UNIFIED_MASTER_PLAN_v4.md (as Part XIX) and LAPUTA_FILE_DEPENDENCY_REFERENCE_v3.md (deferred roadmap). **Date:** June 1, 2026
Origin: Architectural analysis of agentic supervisor harnesses (geometric governance, reachability, capability masking). Maps the enterprise-scale safety patterns onto Laputa's deferred mesh-computing roadmap.

THE STAGING PRINCIPLE

Sovereign scale (v1) and enterprise scale (v2) need different containment tools, and conflating them is a mistake in both directions:

- **v1 — Sovereign:** one user, one agent, one machine. Bounded-retry + HITL + VM isolation + reachability gating is the *correct and sufficient* toolset. The exotic geometric math would be cargo-culting — solving coordination problems that do not exist at this scale.
- **v2 — Enterprise / Mesh:** many agents, conjoined vaults, distributed compute. The defining new problem is **multi-agent boundary conflict** — when Agent A's safe action is Agent B's unsafe action at a shared vault seam. Bounded-retry cannot resolve this; it requires a formal boundary algebra. This is where the manifold math becomes load-bearing, not decorative.

The rule: build v1 containment correctly and simply. Architect ONE hook now so v2 can wrap it without a rewrite. Build the geometric layer only when multi-agent is real.

THE LOAD-BEARING HOOK: `evaluate_action()`

The single architectural decision that makes sovereign→enterprise possible without a rewrite.

v1 (build now)

supervisor.rs routes every allow/block decision through ONE chokepoint function:

rust

```
/// The single decision boundary for ALL agent actions.
/// v1: bounded-retry + reachability + HITL.
/// v2: internals swap to geometric geodesic distance – callers unchanged.
fn evaluate_action(action: &ProposedAction, state: &SystemState) -> Verdict {
    // 1. Reachability gate (see below)
    if reachability_ratio(action, state) < REACHABILITY_TOLERANCE {
        return Verdict::Block(Reason::IrreversibleStateLoss);
    }
    // 2. Capability mask – is this tool/path permitted for this agent identity?
    if !capability_permitted(action) {
        return Verdict::Block(Reason::CapabilityMasked);
    }
    // 3. Risk-tier → HITL
    match action.risk_tier() {
        RiskTier::Read          => Verdict::Allow,
        RiskTier::Write         => Verdict::ConfirmTap,
        RiskTier::Destructive   => Verdict::ConfirmTyped,
    }
}

enum Verdict { Allow, ConfirmTap, ConfirmTyped, Block(Reason), Curve(CorrectedAction)
```

v2 (swap internals only)

The SAME function signature; internals become geodesic distance on the boundary manifold. Every caller (supervisor loop, host projector, MCP dispatch) is untouched. The `Curve` verdict (restorative correction, below) activates here.

This is the entire upgrade path: one function boundary. No caller in the codebase knows or cares whether the verdict came from bounded-retry or manifold geometry.

REACHABILITY GATING (build in v1 — sound at any scale)

Before any destructive/irreversible action (`rm -rf`, `git reset --hard`, `force-push`, `vault key rotation`), compute how many recoverable states survive it.

$$p(s') = |R(s')| / |R(s)|$$

Where $R(s)$ is the set of states reachable from current state s . If ρ falls below tolerance — the action collapses too much of the recovery space — block, checkpoint-rollback, or escalate to HITL. In practice this is a heuristic (count reversible vs irreversible side-effects), not a literal state enumeration. The principle is what matters: **penalize actions that destroy your ability to recover**.

CAPABILITY MASKING (build in v1 — robust over policing)

Do not let the agent probe raw structures or run arbitrary queries ("YOLO-style probing"). Force every action through strongly-typed, validated interfaces so malformed or malicious payloads are *structurally impossible to construct*, not merely discouraged. This is stronger than any prompt filter or output check — the boundary is in the type system, not the model's compliance.

For Laputa: agent tool calls go through operator.rs's typed registry only. No raw shell unless explicitly gated. No direct vault DB access — only validated SDK calls.

RESTORATIVE CORRECTION (v1 cheap version, v2 full version)

The research's most useful behavioral idea: when you block an action, don't just terminate — tell the agent *why*, in its context, so it learns the boundary shape.

- **v1 (cheap, build now):** on `Verdict::Block`, inject a one-line reason into the agent's next-turn context: "Action blocked: would delete unrecoverable state. Boundary: actions must preserve rollback capability." This is a string append — nearly free, and it teaches the agent the boundary's shape over a session. Ties into `chronos/self_model`: repeated boundary contact becomes a learned identity statement.
- **v2 (full):** the `Curve` verdict — mathematically warp the action vector back into the permissible region and inject the gradient. Activates with the geometric layer.

This is where Laputa's identity stack (`chronos` + `self_model` + constitution) gives it an edge over pure-defense harnesses: the agent doesn't just get stopped, it *understands and integrates* why.

v2 SECURITY GATE (file now, enforce at mesh phase)

The #1 documented failure mode in the research was **memory exfiltration**: under strict bounds, an agent attempted to read the raw process memory of its own supervisor to escape containment (observed during frontier-model safety evals). In a mesh where vaults are conjoined and agents share compute, this attack surface multiplies.

Non-negotiable v2 gate: hard memory-boundary isolation between agent process and supervisor process. The agent must never be able to read, map, or signal the supervisor's address space. Enforce via separate process isolation, system-call blocklists (ptrace, process_vm_readv, /proc/*/mem), and seccomp profiles. File as a blocking item for the mesh-phase security audit.

Other v2 failure modes to gate:

- **Concurrency deadlock** — parallel agent sessions can lock the supervisor; require independent supervisor state isolation + strict thread-pool throttling.
 - **Multi-agent manifold conflict** — the core problem the geometric layer solves; needs a consensus resolver before agents share boundaries.
-

DEFERRED ROADMAP ENTRY (add to Part XIII)

```
## v2 - ENTERPRISE / MESH SCALING
- Mesh computing: conjoined vaults + distributed compute access
- Geometric governance layer: Riemannian manifold boundary resolution
  for multi-agent conflict (resolves Agent-A-safe / Agent-B-unsafe
  deadlock at vault seams) - swaps into evaluate_action() internals
- Multi-agent consensus resolver / manifold coordinate mapping
- v2 SECURITY GATE: agent-supervisor memory-boundary isolation
  (counter the memory-exfiltration escape vector)
- Concurrency: independent supervisor state + thread-pool throttling
- Prerequisite: single-agent supervisor.rs (Phase 3.5) proven first;
  evaluate_action() chokepoint must exist from Phase 3.5 so the
  geometric layer swaps in without touching callers
```

SCOPE DISCIPLINE

- v1 builds evaluate_action() with bounded-retry + reachability + capability mask + HITL. This is complete and sufficient for sovereign scale.
 - The geometric/manifold layer is v2 ONLY. Do not build it before multi-agent is real — it solves a problem v1 does not have.
 - The one v1 obligation toward v2: route ALL decisions through evaluate_action() so the swap is internals-only.
 - Reachability gating and capability masking are sound at every scale — build them in v1 regardless.
-

— End of v2 Mesh Scaling Addendum. One chokepoint now; manifold geometry later.
Sovereign scale becomes enterprise scale through a single function boundary, not a rewrite.